NETWORKPEER_COMPLETE_GUIDE

NetworkPeer (GigGrid) — Complete Technical Guide

Written August 2026. Beginner-friendly, but goes deep: every technology, every folder, every important file, how the money and jobs flow, what was deployed, and what is left.

---

1. What is NetworkPeer?

NetworkPeer is a **field-work marketplace**. A **client** posts a job (for example "inspect the storefront at this address and upload photos"), and a **worker** near that location accepts it, goes there, captures evidence (photos), and gets paid — but only after the money is safely held in escrow and the work is approved.

The central business rule the whole system is built around:

> A worker may see the private job details and do the work **only after** the client funds an escrow account and the database accepts the exact lifecycle transition.

There are three kinds of users (roles):

Role	Can do
`CLIENT`	Create jobs, fund escrow, approve completed work, view wallet

Role	Can do
`WORKER`	Find nearby funded jobs, accept one atomically, track progress, upload evidence, submit work
`ADMIN`	Verify workers, suspend users, override stuck jobs, view audit log + analytics

The most important design decision: **the browser is only a screen**. The browser never decides who wins a race, never decides whether a payment happened, and never decides whether evidence is real. PostgreSQL (the database) is the source of truth for all of it.

---

2. How the app works — the plain-English story

1. **Sign in.** A phone number + one-time password (OTP). In development the OTP is shown on screen; in production it is sent by SMS (Twilio).
2. **Client creates a job.** Title, description, budget, a location on the map, and a checklist of required evidence. The job is created in status `FUNDING` — **invisible to workers** until money is held.
3. **Client funds escrow.** The backend asks the payment provider (Stripe) to hold the money. When Stripe confirms via a **webhook** (an HTTP call from Stripe to our API), the database records the escrow and the job becomes `POSTED` (visible to workers).
4. **Worker searches nearby.** A worker's saved location is compared to job locations using PostGIS (geography search). Results hide the client's identity and exact address until acceptance.
5. **Worker accepts.** The database function `accept_job` locks the job row and validates everything in **one transaction**. If two workers click accept at the same time, exactly one wins — the other gets a `409 Conflict`.
6. **Worker does the work.** Status moves `ASSIGNED → EN_ROUTE → AT_LOCATION → IN_PROGRESS`. The worker uploads photos as evidence.
7. **Evidence is verified.** The backend reserves the file metadata *before* upload (allowed type, exact size, SHA-256 checksum), the browser uploads directly to S3 via a short-lived signed URL, then the backend re-checks the stored S3 version against the reservation.
8. **Worker submits → client approves.** The client reviews and clicks *Approve and release payout*. The database creates immutable double-entry ledger postings (release escrow, platform fee, worker payout) and dispatches the payout.

9. **Everyone sees the result** — job statuses update live in the browser through Socket.IO, and the wallet screens show server-calculated balances.

---

3. Technology stack — what each piece is and where it lives

Technology	What it is (plain English)	Where it is used in the repo
TypeScript	JavaScript with types, so mistakes are caught before running	All backend + frontend code
Node.js	The JavaScript runtime that runs the backend	`NetworkPeer-main`
Fastify	A fast web framework for Node: routes, JSON validation, plugins	`src/index.ts`, `src/routes/*`
PostgreSQL 16	The database — the single source of truth	`NetworkPeer-main` (via `src/db.ts`, `migrations/*`)
PostGIS	A PostgreSQL extension for location/geography queries	`migrations/001...`, nearby-job search
Redis	An in-memory key/value store — used for OTP codes, rate limits, refresh-token tracking, and BullMQ queues	`src/db.ts`, `src/auth.ts`, background worker
BullMQ	A job queue that runs work in the background (Redis-backed)	`src/services/background-queue-service.ts`

Technology	What it is (plain English)	Where it is used in the repo
Socket.IO	Real-time push from server to browser (live status updates)	`src/services/realtime-hub.ts`, frontend `realtime-sync-bridge.tsx`
JWT (HS256)	Signed “tokens” the browser sends to prove who it is	`src/auth.ts`, `src/middleware/auth.ts`
Zod	Validates incoming request bodies at runtime	`src/routes/*`, `src/contracts.ts`
S3 (AWS)	Object storage for evidence photos, with versioning + encryption	`src/services/media-storage-service.ts`
Stripe (Connect)	Payment provider that holds escrow and pays out workers	`src/services/payment-gateway-service.ts`, `payment-webhooks.ts`
Twilio	Sends the OTP SMS in production	`src/services/sms-provider.ts`
Sentry	Error monitoring (optional)	`src/observability.ts`
React + TanStack Start/Router	The frontend framework (browser UI + server rendering)	`NetworkPeer-platform-main/src`
Vite / Nitro	Build tooling; produces the deployable frontend bundle	`vite.config.ts`, `.output/`
Docker / docker compose	Packages the backend so it runs anywhere	`Dockerfile`, `docker-compose.yml`, `docker-compose.prod.yml`
GitHub Actions	Automated checks (CI) on every push	`.github/workflows/deploy.yml`

Technology	What it is (plain English)	Where it is used in the repo
Cloudflare quick tunnels	Gives the local server a public HTTPS URL (demo tool)	scripts/start-demo.sh

---

4. Architecture

```
Browser (Vercel / Nitro)              Fastify API (Node.js)
  VITE_API_BASE_URL  ---------------->  REST + Socket.IO
  Bearer access token                   Helmet + CORS + rate
limits (Redis)
  Presigned S3 POST  ---------------->  PostgreSQL functions /
PostGIS
                                             |
                                             +--> PostgreSQL
durable outboxes + ledger
                                             +--> Redis / BullMQ
background workers
                                             +--> S3 versioned
evidence
                                             +--> Stripe Connect
webhooks
                                             +--> FCM push
(optional)
                                             +--> Sentry +
structured logs
```

The data-authority rule: PostgreSQL is authoritative for jobs, funding, evidence acceptance, ledger postings, push state, and queue state. Redis is *not* a source of truth — it is a fast cache/queue. If Redis is restarted, the API and workers recover outstanding work from PostgreSQL (this is why background jobs are written to a PostgreSQL “outbox” first, then relayed to Redis).

Least-privilege database roles: the app does not connect to the database as an all-powerful owner. Four restricted roles exist, each with narrowly granted functions:

Runtime variable	Database role	Purpo
`DATABASE_URL`	`networkpeer_app`	Norma reques

Runtime variable	Database role	Purpo
`DATABASE_ADMIN_URL`	`networkpeer_admin_api`	Admin backo
`DATABASE_MEDIA_VERIFIER_URL`	`networkpeer_media_verifier`	Evider verific
`DATABASE_FINANCIAL_URL`	`networkpeer_financial_api`	Financ operat

The dangerous database functions are `SECURITY DEFINER` — they run with elevated rights only for their narrow purpose, and the application role cannot call anything else.

---

5. The codebase, top to bottom

5.1 Repository layout

```
NETWORKPEER/                      <- one Git repository
├── .github/workflows/deploy.yml <- CI: lint, typecheck, build,
unit + E2E, Docker
├── NetworkPeer-main/             <- BACKEND (Fastify API + worker
+ database)
│   ├── src/                      <- TypeScript source
│   ├── migrations/               <- 38 SQL migrations (schema, in
order)
│   ├── scripts/                  <- migrate, provision, verify,
demo helpers
│   ├── tests/                    <- unit tests + E2E test
│   ├── Dockerfile                <- multi-stage build for
production
│   ├── docker-compose.yml        <- local PostGIS + Redis
│   ├── docker-compose.prod.yml   <- production-like full stack
│   └── .env                      <- LOCAL secrets (gitignored,
never commit)
└── NetworkPeer-platform-main/    <- FRONTEND (React / TanStack
Start)
    ├── src/routes/               <- every page of the app
    ├── src/lib/                  <- API client, session, helpers
    ├── src/components/           <- UI components (incl. realtime
bridge)
    └── vite.config.ts            <- build configuration
```

5.2 Backend entry points

File	What it does
`src/index.ts`	Starts Fastify: CORS, Helmet, rate limits, all routes, Socket.IO hub, graceful shutdown. Exports `buildApp` so tests can boot the server in-process.
`src/background-worker.ts`	A separate process that runs background queues + payment dispatch without serving HTTP. Scaled independently in production.
`src/config.ts`	Reads + validates **every** environment variable with Zod. Enforces production rules (e.g. rejects `OTP_ECHO_IN_RESPONSE=true`, rejects weak JWT secrets, requires `sslmode=require` URLs, requires real S3 bucket name).
`src/db.ts`	Owns the PostgreSQL pools (normal, admin, media, financial) and Redis.
`src/contracts.ts`	TypeScript types + the standard API envelope `{ success, data, error }` (this is why every response looks the same).
`src/repository.ts`	The only file that talks raw SQL to PostgreSQL. Everything the API does with data goes through here.
`src/state-machine.ts`	The pure “legal transitions” table (e.g. `FUNDING → POSTED → ASSIGNED → …`) that the application enforces — the database enforces the same rules with constraints, so the two must agree.
`src/auth.ts`	OTP hashing, JWT signing/verification with constant-time comparison, refresh-token family rotation, Redis rate-limit helpers.

File	What it does
`src/middleware/auth.ts`	`requireAuth` / `requireRole` — checks the bearer JWT and reloads the active user on every request.
`src/observability.ts`	Sentry + Pino structured logging setup.

5.3 Backend routes (the API surface)

Route file	Endpoints	What it's for
`routes/system.ts`	`GET /live`, `GET /health`	Liveness + database/PostG health probes
`routes/auth.ts`	`POST /auth/otp/request`, `/auth/otp/verify`, `/auth/refresh`, `/auth/logout`, `GET /auth/me`	Phone sign-in, tokens, session logout revokes with a refresh token even afte access expiry, while a valid bearer remains subject-bound
`routes/client-jobs.ts`	create / list / detail / cancel jobs	Client job management
`routes/worker-jobs.ts`	`GET /worker/jobs/nearby`, `GET /worker/jobs/:id`, `POST /worker/jobs/:id/accept`	Discovery + atomic acceptance
`routes/work.ts`	status, upload URL, evidence confirm, submit	Worker task lifecycle + evidence
`routes/financial.ts`	fund escrow, approve work, client/worker wallets	Money flows
`routes/payment-webhooks.ts`	`POST /webhooks/payments`	Stripe calls this to confirm payments

Route file	Endpoints	What it's for
`routes/sync.ts`, `routes/worker-sync.ts`	cursor-based sync	Durable event recovery after reconnect
`routes/notifications.ts`	list/read/read-all, device registration	In-app notifications
`routes/admin.ts`	audit log, job overrides, user management, analytics	Backoffice
`routes/admin-workers.ts`	worker verification	Admin verifies workers

5.4 Backend services (business logic)

Service	What it does
`auth-service.ts`	Creates/looks up users after OTP verification, issues token pairs
`otp-service.ts`	OTP TTL, attempt limits, delivery
`sms-provider.ts`	Console (dev) vs Twilio (prod) SMS boundary
`job-service.ts`	Client job policy above raw SQL
`worker-job-service.ts`	Verified-worker discovery, privacy-safe projections, acceptance policy
`media-storage-service.ts`	S3: presigned POST policy, `HeadObject`, tagging, bucket readiness checks
`work-evidence-service.ts`	Worker progress, evidence reservation/confirmation, version pinning, submit validation
`notification-service.ts`	Cursor sync, notification paging/read state, device tokens
`realtime-hub.ts`	Authenticates Socket.IO connections, joins user rooms, listens to PostgreSQL `NOTIFY`, emits events

Service	What it does
`push-notification-service.ts`	Durable FCM push delivery worker (optional)
`ledger-service.ts`	Funding, approval, payout dispatch/retry, webhook settlement, wallet summaries
`payment-gateway-service.ts`	`StubPaymentGateway` (dev) + `StripePaymentGateway` (prod), raw webhook signature verification, and a `signPaymentWebhook` helper used by the E2E + demo script
`payment-dispatch-service.ts`	PostgreSQL-leased retry dispatcher for interrupted payouts
`background-queue-service.ts`	BullMQ relay/workers for durable media + push outboxes

5.5 Migrations (the schema history)

Migrations are applied **in order**, exactly once, with SHA-256 checksums, guarded by an advisory lock. They are forward-only: you never edit an applied migration; you add a new one.

Group	Migrations	What they build
Phase 1–5 core	`001` – `016`	PostGIS + enums, users/roles, jobs, lifecycle constraints, verified-worker admission, geospatial search, evidence reservations + S3 version pinning, secure function search paths
Phase 6–7 sync + admin	`017` – `020`	Durable sync outbox, notifications, device tokens, commit-ordered cursors, append-only audit log, admin controls, suspension

Group	Migrations	What they build
Phase 8 financial	`021` – `035`	Fund-before-publish lifecycle, ledger accounts/transactions, payment operations + webhook inbox, immutable postings, escrow settlement, zero-fee handling, suspension freezing
Phase 8–9 hardening	`036` – `039`	Payout reversal compensation, leased dispatch retry, media-processing outbox, name-resolution fixes, durable FCM retry due times

5.6 Scripts and tests

File	Purpose
`scripts/migrate.ts`	The migration runner
`scripts/provision-app-role.sql`	Creates the four least-privilege database roles + grants
`scripts/provision-admin.sql`	Bootstraps one pre-approved ADMIN account (never elevates an existing user)
`scripts/verify-phase1.ts` … `verify-phase9.ts`, `verify-auth-security.ts`	Live verifiers for each phase (schema, auth rotation, geography, evidence, sync, admin, financial, queues)
`scripts/simulate-payment-webhook.mjs`	**Demo tool:** signs + posts a fake `payment_intent.succeeded` webhook so funding works without Stripe
`scripts/start-demo.sh` / `stop-demo.sh`	One-command demo launcher / stopper (added for this presentation)

File	Purpose
`scripts/migrate-and-provision.sh`	Combined migration + provisioning helper for deploy
`tests/auth.test.ts`, `tests/state-machine.test.ts`	Fast unit tests (`npm test`)
`tests/e2e/core-job-acceptance.e2e.test.ts`	The full E2E: create job → fund → signed webhook → publish → two workers race → exactly one wins (`npm run test:e2e`, requires a database named with `test` / `e2e`)

5.7 Docker

File	What it does
`Dockerfile`	Multi-stage: builds TypeScript, then runs `dist/index.js` (runtime image). A separate `migrator` target runs migrations/provisioning one-shot.
`docker-compose.yml`	Local dev: PostGIS (port 5433) + Redis (6379).
`docker-compose.prod.yml`	Production-like local stack: PostGIS, password-protected Redis, one-shot migration service, API, separate worker. Requires `.env.prod` with generated secrets.

5.8 Frontend (NetworkPeer-platform-main)

Area	Files	What it does
Bootstrap	`src/start.ts`, `src/server.ts`, `src/router.tsx`, `src/routeTree.gen.ts`	TanStack Start bootstrap, SSR err generated route manifest
API client	`src/lib/api.ts`	REST client: envelope parsing, be cursor sync, notifications

Area	Files	What it does
Session	`src/lib/auth-session.ts`	Per-tab session store (sessionSto intentionally tab-scoped)
Realtime	`src/components/realtime-sync-bridge.tsx`	Socket.IO connection, durable cu invalidation, live toasts
Pages	`src/routes/*.tsx`	Every screen: landing, auth (OTP) (dashboard/jobs/new/detail/notific (discovery/job/task/profile/wallet), (dashboard/analytics/clients/jobs/
UI kit	`src/components/ui/*`	Reusable components (buttons, c

Which frontend screens are live vs prototype: authentication, the notification inbox, and the realtime bridge are fully wired. Client job create/detail/funding, worker discovery/acceptance, the live task + evidence screens, and the wallet screens were rewired to the real API for this presentation. Admin pages, client evidence review, browser push (FCM), and private-route guards remain prototypes (see §8).

---

6. Key flows in technical detail

6.1 Sign-in and tokens

1. Browser calls `POST /auth/otp/request` with a phone number. The OTP is stored in Redis with a TTL and attempt limits.
2. Browser calls `POST /auth/otp/verify` with the code (dev mode: the code is echoed in the response; production: Twilio SMS, no echo).
3. A new user is created with role `CLIENT` or `WORKER` through `register_otp_user` (a `SECURITY DEFINER` function), and the API returns **two JWTs**: a short-lived **access token** (15 min) and a **refresh token** (7 days).
4. The refresh token is single-use: every refresh rotates it, and replaying an old token revokes the whole token family (stops stolen tokens being reused).
5. On a `401`, the browser refreshes once and retries the request. Only a failed refresh logs the user out.

6.2 Funding and escrow (the money hold)

1. `POST /client/jobs` creates the job with status `FUNDING`. It is invisible to workers.

2. `POST /client/jobs/:id/fund` creates a **payment operation** with a stable idempotency key (retrying never double-charges) and asks the payment gateway to create a payment intent (stub: `stub_pi_…`; Stripe: real PaymentIntent).
3. The provider eventually calls `POST /webhooks/payments`. The API verifies the HMAC signature, matches the operation, and — only inside one database transaction — records the escrow journal entry and moves the job to `POSTED`.
4. The browser never infers success locally; it just refreshes and reads the new status.

6.3 Atomic acceptance (the concurrency proof)

`POST /worker/jobs/:id/accept` runs `accept_job(worker_id, job_id)` in PostgreSQL: it locks the job row, checks the worker is verified, eligible, and not on another job, then assigns. Because of the row lock, two simultaneous clicks serialize in the database: one commits, the other gets `409`. *This is the demo's showpiece: the browser race is impossible.*

6.4 Evidence (the S3 dance)

1. `POST /work/upload-url` **reserves** the evidence: content type, exact byte size, SHA-256 — and returns a short-lived presigned POST for the bucket.
2. The browser uploads the file **directly to S3** with that signed form (it never holds S3 credentials).
3. `POST /work/evidence` makes the API read the stored object's metadata via `HeadObject` (exact version ID, ETag, size, checksum, lifecycle tag) and verify it against the reservation — inside the database.
4. Only `UPLOADED` / `VERIFIED` evidence for every required checklist item allows `POST /work/submit`.

6.5 Ledger and payout (the money movement)

Approval calls a database function that, in one transaction: releases escrow, records a platform fee, creates a **pending payout** journal entry, and dispatches the external payout. Everything is **double-entry** (debits = credits) and **immutable** — balances are never stored, they are *projections* computed from the completed postings. Payouts move to final state only when the provider webhook confirms.

6.6 Durable sync + realtime (why nothing is lost)

1. A job/evidence/notification change fires a PostgreSQL trigger inside the same transaction: it writes a recipient-specific `sync_events` row (durable outbox) and `pg_notify`s.

2. API instances listening on PostgreSQL `NOTIFY` push the event to the right user's Socket.IO room.
3. The browser applies the live event, then **reconciles** with `GET /sync?cursor=…` until `has_more: false` — so even missed events are recovered from the database.
4. The cursor is a PostgreSQL `BIGINT` kept as a decimal string (never a JS `Number` — precision!).

---

7. What was done for this deployment (this session, verified)

Item	Status
Backend `typecheck`, `lint`, `build`, unit tests	✅ all pass
Funded atomic-acceptance E2E (create → fund → signed webhook → publish → 2-worker race)	✅ passed against local `networkpeer_e2e` DB
Fixed **OTP signup bug** — `register_otp_user` was called with arguments swapped, so every new user failed with `invalid input value for enum user_role: "Unnamed user"`	✅ fixed in `src/repository.ts`, verified live
Fixed **CORS** — frontend dev server runs on port 8080 but the API only allowed 3001/5173	✅ added 8080 to `.env`
Restarted the API — the old process was 4 days stale and `/live` 404'd	✅ fresh build running on port 3000
Added `scripts/simulate-payment-webhook.mjs` so the demo's funding step needs **no Stripe account**	✅ verified the signature is accepted
Added `scripts/start-demo.sh` / `stop-demo.sh` — one command for tomorrow	✅
Installed `cloudflared` and verified a public quick tunnel (`/live` → 200	✅

Item	Status
through `https://…trycloudflare.com`)	
Committed everything + pushed to `github.com/addy9087/Networkpeer`	✅ `main`
GitHub Actions CI (lint → typecheck → build → unit → E2E → Docker)	✅ triggered, runs on every push

---

8. Deployment: what's live and what remains

8.1 Tonight — live demo from this Mac (works now)

`./scripts/start-demo.sh --public` starts infra + API + frontend + Cloudflare tunnels and prints the public URLs. The presentation laptop only needs a browser (normal window = client, incognito = worker). Full walkthrough: `docs/DEPLOY_FOR_PRESENTATION.md`.

8.2 Tomorrow — always-on cloud deployment (from GitHub)

1. **Backend on Railway or Render**: connect the GitHub repo, root directory `NetworkPeer-main`; two services (API: `node dist/index.js`, worker: `node dist/background-worker.js`); managed PostgreSQL (PostGIS) + Redis (`rediss://`).
2. **Run migrations + provisioning once** with the migration-owner URL before traffic: `npm run migrate` + `scripts/provision-app-role.sql`.
3. **Frontend on Vercel**: root `NetworkPeer-platform-main`, Node 24, `NITRO_PRESET=vercel`, `VITE_API_BASE_URL=https://<api>/api/v1`, `VITE_API_PREFIX=/api/v1`.
4. Set backend `CORS_ORIGINS` to the exact Vercel origin.
5. Set the full environment table from `EXECUTIVE_PRESENTATION_AND_DEPLOYMENT_GUIDE.md` §2.5.

8.3 What still needs doing (in priority order)

Item	Why	Effort
S3 bucket for evidence	Evidence upload (demo Step C) needs a versioned, encrypted private bucket + IAM keys; without it, skip	~10 min (AWS) / 1 h (MinIO code change)

Item	Why	Effort
	that step or wire a local MinIO endpoint	
Stripe test keys + webhook	Required only for the cloud deploy (production rejects the stub gateway); the local demo uses the webhook simulator	~20 min
Twilio account	**Required for cloud deploy** — production rejects `OTP_ECHO_IN_RESPONSE=true` and `SMS_PROVIDER=console`, so login needs real SMS	~15 min
Provision admin + verify worker	One-time SQL (`scripts/provision-admin.sql` + `admin_set_worker_verification`) with your real demo phone numbers	5 min
Private-route guards	`/client`, `/worker`, `/admin` pages don't redirect unauthenticated users yet	1–2 h
Admin UI integration	Admin screens still use mock data; APIs exist	2–4 h
Client evidence review UI	Review/approve screen still prototype	1–2 h
Firebase browser push	FCM service worker + permission flow; backend support exists	2–4 h
Stripe Connect recipient onboarding / refunds	Worker payout recipients + frozen-escrow dispute handling	1–2 days
Observability hardening	Real Sentry DSN, alerting, log retention in production	2–4 h

---

9. Glossary (beginner terms)

Term	Meaning
API	The backend's "phone line": the browser sends HTTP requests, the server responds with JSON
Endpoint / route	One specific API call, e.g. `POST /api/v1/client/jobs`
Webhook	A third party (Stripe) calling our API to tell us something happened (payment succeeded)
Escrow	Money held by a neutral third party until conditions are met
Ledger / double-entry	Accounting where every entry has a matching opposite entry; totals always balance
JWT	A signed, readable-in-JSON token that proves who you are; HMAC-signed so it can't be forged
OTP	One-time password (the 6-digit SMS code)
Presigned URL / POST	A short-lived, pre-authorized link that lets a browser upload a file straight to S3
SHA-256	A checksum: a fixed-length "fingerprint" of a file used to prove it didn't change
ETag / VersionId	S3's fingerprints of an object; VersionId pins the exact stored version
PostGIS	Spatial extension that makes "jobs within 5 km of me" a database query
Outbox	A table of "things to do" written in the same transaction as the change, so nothing is lost

Term	Meaning
Cursor	A bookmark saying “I’ve seen events up to here”, used to catch up after a disconnect
CORS	A browser security rule: a page from origin A may only call origin B if B explicitly allows it
CI	Continuous Integration — automated checks that run on every code push
Migration	A versioned SQL file that changes the schema step by step
SSR	Server-Side Rendering — the frontend server prints the HTML first, so pages load fast
Idempotency key	A unique key so retrying a request never creates a duplicate (no double charge)